

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería de Software

Consumo de Web Services REST desde un Front-end AlpineJS — Proyecto ArtiSync

Asignatura: Aplicaciones Web (5to nivel) — Segundo Corte

Integrantes:

Figueroa Morales Bryan Javier
Llor Medranda Marlon Taylor

Docente:

Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Julio 2026

Repositorio: [https://github.com/011011bjf/
GA-Exposici-n-Segundo-Corte-Frameworks-del-Back-End-](https://github.com/011011bjf/GA-Exposici-n-Segundo-Corte-Frameworks-del-Back-End-)

1. Introducción

ArtiSync nació como Proyecto Fin de Curso (PFC) durante el Primer Corte de la asignatura, con un backend en Spring Boot que expone la lógica de negocio y la persistencia de datos a través de una API REST. Este Segundo Corte extiende ese mismo framework de trabajo agregando una capa de presentación construida con AlpineJS, cuyo único propósito es consumir los web services ya expuestos por ese backend: autenticación de usuarios (incluyendo verificación en dos factores), y administración de cuentas mediante operaciones CRUD. Es importante aclarar que dicho backend pertenece al PFC de Bryan Figueroa como parte del mismo equipo de trabajo del curso, y no corresponde a una API de un tercero externo ajeno a esta entrega; el frontend documentado aquí y el backend que consume forman parte de un mismo esfuerzo conjunto.

Desde el punto de vista arquitectónico, el consumo de la API sigue el estilo REST descrito originalmente por Fielding [3], donde cada recurso (usuarios, sesiones, credenciales) se manipula mediante los verbos HTTP estándar sobre representaciones JSON, sin mantener estado de sesión en el servidor más allá del token emitido. La calidad del software resultante se enmarca en el modelo ISO/IEC 25010 [4]: al depender de un contrato JSON explícito y estable entre cliente y servidor (documentado en la Sección 7), este trabajo aporta principalmente a la característica de *compatibilidad/interoperabilidad* del modelo, más que a otras características como el rendimiento o la portabilidad.

El objetivo de este documento es explicar, de forma reproducible, cómo el frontend AlpineJS consume la API REST del backend: el flujo de login con verificación 2FA y el CRUD completo de usuarios, con fragmentos de código reales extraídos del repositorio. Queda fuera de alcance el detalle de implementación interno del backend (controladores, servicios, persistencia), ya que ese componente se documenta en su propio repositorio como parte del mismo PFC. Finalmente, siguiendo las prácticas de transparencia en el desarrollo de software que recomienda el SWEBOK v4.0 [7], el uso de asistencia de inteligencia artificial durante la elaboración de este proyecto y de este informe se declara explícitamente en la Sección 3.

2. Paso a paso del consumo de la API REST

Esta sección documenta, con fragmentos de código extraídos directamente de los archivos fuente del repositorio (sin reescribir ni parafrasear), cómo el frontend AlpineJS consume la API REST del backend Spring Boot de ArtiSync. Todos los verbos HTTP y códigos de estado citados siguen la se-

mántica definida en [2], y todos los cuerpos de petición/respuesta se serializan en formato JSON según [1].

1. Configuración del cliente HTTP

Todas las peticiones autenticadas del frontend pasan por un único punto de entrada: `api-client.js`. Ahí se define la URL base del backend y el token de acceso se persiste en `localStorage` bajo la clave `accessToken`, siguiendo el esquema de token portador (*Bearer*) de [5]:

Listing 1: Constante `API_BASE_URL` (`api-client.js`, línea 1)

```
1 const API_BASE_URL = 'http://localhost:8080/api';
```

Listing 2: Objeto `authStore`: lectura, escritura y borrado del token en `localStorage` (`api-client.js`, líneas 3–13)

```
3 const authStore = {
4   getToken() {
5     return localStorage.getItem('accessToken');
6   },
7   setToken(token) {
8     localStorage.setItem('accessToken', token);
9   },
10  removeToken() {
11    localStorage.removeItem('accessToken');
12  }
13};
```

La función `fetchWithAuth()` envuelve el `fetch()` nativo del navegador: fija por defecto los headers `Content-Type: application/json` y `Accept: application/json` (línea 19–20), y si existe un token guardado, agrega el header `Authorization: Bearer <token>` (líneas 24–26) antes de resolver la URL completa concatenando `API_BASE_URL` con la ruta relativa recibida (línea 33):

Listing 3: Función `fetchWithAuth()` (`api-client.js`, líneas 15–45)

```
15 async function fetchWithAuth(url, options = {}) {
16   const token = authStore.getToken();
17
18   const headers = {
19     'Content-Type': 'application/json',
20     'Accept': 'application/json',
21     ...options.headers,
22   };
```

```

23
24     if (token) {
25         headers['Authorization'] = 'Bearer ${token}';
26     }
27
28     const config = {
29         ...options,
30         headers
31     };
32
33     let response = await fetch(`${API_BASE_URL}${url
34         }`, config);
35
36     if (response.status === 401) {
37         // Podríamos intentar refresh token aquí,
38         // pero para mantenerlo sencillo:
39         authStore.removeToken();
40         // Redirigir al login si no estamos ya allí
41         if (!window.location.pathname.includes('login
42             .html')) {
43             window.location.href = 'login.html';
44         }
45     }
46
47     return response;
48 }

```

2. Autenticación inicial: POST /auth/login

Archivo: login.html. El formulario de inicio de sesión envía el correo y la contraseña capturados por Alpine (`x-model="form.correo"` / `x-model="form.contrasena"`) como cuerpo JSON del método POST, conforme a la semántica de creación/procesamiento de recursos descrita en [2]:

Listing 4: POST /auth/login: petición, body y manejo inicial de la respuesta (login.html, líneas 272–289)

```

272         try {
273             const response = await fetch
274                 (`${API_BASE_URL}/auth/
275                 login`, {
276                 method: 'POST',
277                 headers: {

```

```

276         'Content-Type': '
277             application/json',
278         'Accept': '
279             application/json'
280     },
281     body: JSON.stringify(this
282         .form)
283 });
284
285 const data = await response.
286     json();
287
288 if (!response.ok) {
289     if (data.message) {
290         throw new Error(data.
291             message);
292     }
293     throw new Error('
294         Credenciales
295         incorrectas o error en
296         el servidor');
297 }

```

La respuesta se interpreta según el contrato del backend: si el campo `requiere2fa` viene en `true`, el flujo normal de login se detiene y se activa el formulario de verificación en dos pasos (ver Paso 8) en lugar de tratarlo como un error:

Listing 5: Interpretación del campo `requiere2fa` de la respuesta (`login.html`, líneas 291–294)

```

291         if (data.requiere2fa) {
292             this.mostrarCampo2FA =
293                 true;
294             return;
295         }

```

3. Verbo GET con autenticación: listado paginado de usuarios

Archivo: `crud-usuarios.html`. La función `cargarUsuarios()` invoca `fetchWithAuth()` con la ruta relativa `/admin/usuarios` y los parámetros de consulta de paginación (`page`, `size`, `sortBy`, `direction`). Al no especificarse un segundo argumento de opciones, `fetchWithAuth()` usa GET por defecto y agrega automáticamente el header `Authorization`

descrito en el Paso 1 sin que este código tenga que construirlo manualmente:

Listing 6: GET /admin/usuarios paginado vía fetchWithAuth() (crud-usuarios.html, líneas 331–347)

```
331         async cargarUsuarios() {
332             this.cargando = true;
333             try {
334                 const res = await
                    fetchWithAuth('/admin/
                        usuarios?page=${this.page
                        }&size=${this.size}&sortBy
                        =idUserio&direction=desc
                        ');
335                 if (res.ok) {
336                     const data = await res.
                        json();
337                     this.usuarios = data.
                        content;
338                     this.totalPages = data.
                        totalPages || 1;
339                 } else {
340                     console.error('Error
                        cargando usuarios',
                        await res.text());
341                 }
342             } catch (e) {
343                 console.error('Network error
                        ', e);
344             } finally {
345                 this.cargando = false;
346             }
347         },
```

4. Verbo PUT: edición de usuario

Archivo: crud-usuarios.html. La función guardarEdicion() envía la representación completa del recurso editado a PUT /admin/usuarios/{id}, consistente con la semántica de reemplazo idempotente de un recurso que define [2] para el método PUT:

Listing 7: PUT /admin/usuarios/{id}: edición de usuario (crud-usuarios.html, líneas 412–434)

```

412         async guardarEdicion() {
413             try {
414                 const payload = {
415                     nombres: this.formEdit.
                        nombres,
416                     apellidos: this.formEdit.
                        apellidos,
417                     estadoCuenta: this.
                        formEdit.estadoCuenta,
418                     roles: this.formEdit.
                        roles
419                 };
420                 const res = await
                    fetchWithAuth('/admin/
                        usuarios/${this.formEdit.
                            id}', {
421                     method: 'PUT',
422                     body: JSON.stringify(
                        payload)
423                 });
424                 if (res.ok) {
425                     alert('Usuario modificado
                        con éxito');
426                     this.modalAbierto = false
                        ;
427                     this.cargarUsuarios();
428                 } else {
429                     alert('Error modificando
                        usuario: ' + (await
                            res.text()));
430                 }
431             } catch (e) {
432                 alert('Error de red');
433             }
434         },

```

5. Verbo PATCH: cambio de estado

Archivo: crud-usuarios.html. A diferencia del PUT anterior, `cambiarEstado()` solo modifica un atributo puntual del recurso (`estadoCuenta`), por lo que se usa `PATCH /admin/usuarios/{id}/estado` con un cuerpo mínimo (`{ nuevoEstado }`) en lugar de reenviar el usuario completo:

Listing 8: PATCH /admin/usuarios/{id}/estado: cambio de estado (crud-usuarios.html, líneas 436–452)

```
436         async cambiarEstado(id, nuevoEstado)
437         {
438             if (!confirm('¿Estás seguro de
439                 cambiar el estado a ${
440                     nuevoEstado}?')) return;
441             try {
442                 const res = await
443                     fetchWithAuth('/admin/
444                         usuarios/${id}/estado', {
445                             method: 'PATCH',
446                             body: JSON.stringify({
447                                 nuevoEstado })
448                             });
449                 if (res.ok) {
450                     alert('Estado cambiado
451                         con éxito');
452                     this.cargarUsuarios();
453                 } else {
454                     alert('Error cambiando
455                         estado: ' + (await res
456                             .text()));
457                 }
458             } catch (e) {
459                 alert('Error de red');
460             }
461         },
```

6. Verbo DELETE: eliminación y revocación de sesiones

Archivo: crud-usuarios.html. Existen dos operaciones DELETE distintas: `revocarSesiones()` invalida todas las sesiones activas de un usuario (DELETE /admin/usuarios/{id}/sesiones) sin eliminar la cuenta, y `eliminarUsuario()` da de baja lógicamente al usuario (DELETE /admin/usuarios/{id}). Ambas confirman la acción con el usuario antes de ejecutarla (`confirm(...)`), dado que DELETE es una operación destructiva según [2]:

Listing 9: DELETE /admin/usuarios/{id}/sesiones y DELETE /admin/usuarios/{id} (crud-usuarios.html, líneas 454–485)

```
454         async revocarSesiones(id) {
```

```

455         if (!confirm('¿Revocar todas las
sesiones de este usuario?
Perderá el acceso de inmediato
.')) return;
456     try {
457         const res = await
            fetchWithAuth('/admin/
usuarios/${id}/sesiones',
            {
458             method: 'DELETE'
459         });
460         if (res.ok) {
461             alert('Sesiones revocadas
.');
```

```

        } else {
462             alert('Error revocando
463             sesiones.');
```

```

        }
464     } catch (e) {
465         alert('Error de red');
```

```

    },
466
467
468
469
470     async eliminarUsuario(id) {
471         if (!confirm('¿Seguro que deseas
eliminar a este usuario de
forma lógica?')) return;
```

```

472     try {
473         const res = await
            fetchWithAuth('/admin/
usuarios/${id}', {
474             method: 'DELETE'
475         });
476         if (res.ok) {
477             alert('Usuario eliminado
correctamente.');
```

```

            this.cargarUsuarios();
478         } else {
479             alert('Error eliminando
480             usuario.');
```

```

        }
481     } catch (e) {
482         alert('Error de red');
```

```

    }
483

```

```

484         }
485     }

```

7. Manejo de JSON: serialización y parseo

En todos los pasos anteriores se repite el mismo patrón de serialización descrito en [1]: el cuerpo saliente se construye con `JSON.stringify(...)` (ver Listados 4, 7 y 8), y la respuesta entrante se parsea de forma asíncrona con `await response.json()` antes de leer cualquiera de sus campos (ver Listados 6 y 5). El header `Content-Type: application/json` fijado por `fetchWithAuth()` (Listado 3) le indica al backend cómo interpretar ese cuerpo, y el header `Accept: application/json` le indica en qué formato se espera la respuesta.

8. Autenticación avanzada: verificación 2FA

Archivo: `login.html`. Cuando el login detecta `requiere2fa: true` (Listado 5), se muestra un segundo formulario que pide un código de 6 dígitos. Al enviarlo, `submitCodigo2FA()` llama a `POST /auth/2fa/verify` con el correo ya capturado y el código ingresado; esa segunda respuesta sí trae el `accessToken` completo (token portador según [5]), y a partir de ahí el flujo termina exactamente igual que un login sin 2FA: se guarda el token y se redirige al panel:

Listing 10: `POST /auth/2fa/verify`: verificación del código y finalización del login (`login.html`, líneas 315–355)

```

315         async submitCodigo2FA() {
316             this.error = null;
317             this.cargando = true;
318
319             try {
320                 const response = await fetch
321                     (`${API_BASE_URL}/auth/2fa/verify`, {
322                     method: 'POST',
323                     headers: {
324                         'Content-Type': 'application/json',
325                         'Accept': 'application/json'
326                     },
327                     body: JSON.stringify({
328                         correo: this.form.

```

```

327         correo, codigo: this.
328         codigo2fa })
329     });
330
331     const data = await response.
332     json();
333
334     if (!response.ok) {
335         if (data.message) {
336             throw new Error(data.
337             message);
338         }
339         throw new Error('Código
340         inválido o expirado');
341     }
342
343     // Guardar el token en el
344     storage
345     if (data.accessToken) {
346         authStore.setToken(data.
347         accessToken);
348     }
349
350     this.success = true;
351
352     // Redirección al panel
353     setTimeout(() => {
354         window.location.href = '
355         crud-usuarios.html';
356     }, 1000);
357
358     } catch (err) {
359         this.error = err.message;
360     } finally {
361         this.cargando = false;
362     }
363 }

```

9. Manejo de errores

El manejo de errores ocurre en dos niveles. A nivel de transporte, `fetchWithAuth()` intercepta cualquier respuesta con código 401 (no autorizado, según [2]): limpia el token guardado y redirige a `login.html`,

salvo que ya se encuentre en esa página, evitando así dejar al usuario con un token inválido en `localStorage` — una práctica alineada con el control de gestión de sesiones que recomienda [6]:

Listing 11: Interceptor de 401 en `fetchWithAuth()` (`api-client.js`, líneas 35–42)

```
35     if (response.status === 401) {
36         // Podríamos intentar refresh token aquí,
           pero para mantenerlo sencillo:
37         authStore.removeToken();
38         // Redirigir al login si no estamos ya allí
39         if (!window.location.pathname.includes('login
           .html')) {
40             window.location.href = 'login.html';
41         }
42     }
```

A nivel de formulario, tanto `login.html` (Listado 4, líneas 284–289) como `registro.html` muestran al usuario el mensaje de error que el propio backend envía en el campo `data.message` del cuerpo de la respuesta, con un mensaje genérico de respaldo si ese campo no viene presente:

Listing 12: Manejo de errores con `data.message` / `data.error` (`registro.html`, líneas 418–426)

```
418         if (!response.ok) {
419             // Captura de errores 400
               (ej. Correo ya
               registrado, Errores de
               validación)
420             if (data.message) {
421                 throw new Error(data.
                   message);
422             } else if (data.error) {
423                 throw new Error(data.
                   error);
424             }
425             throw new Error('Ha
               ocurrido un error en
               el servidor al
               registrarse');
426         }
```

3. Conclusiones

A nivel de frontend, se logró implementar un flujo de autenticación completo, incluyendo la verificación en dos factores cuando el backend la exige (Paso 8), y un CRUD de usuarios que cubre los cuatro verbos HTTP principales — GET paginado, PUT de edición, PATCH de cambio de estado y DELETE de eliminación/revocación de sesiones (Pasos 3 a 6) —, todo ello con un manejo centralizado de errores de autorización: cualquier respuesta 401 limpia el token guardado y redirige al login (Paso 9), evitando que la interfaz quede en un estado inconsistente con credenciales inválidas.

La prueba end-to-end contra el backend real **sí se realizó**: con los tres contenedores de Docker (backend, PostgreSQL, Redis) corriendo en estado **healthy**, se ejecutaron peticiones reales — petición y respuesta completas documentadas en la carpeta **evidencia/** del repositorio — contra POST /auth/registro (201 Created), POST /auth/login (200 OK, requiere2fa: false) y los cuatro verbos de /admin/usuarios (GET, PUT, PATCH, POST). Esta última parte dejó una limitación parcial, no total: como el registro público solo permite el rol **CLIENTE** y no se dispone de la contraseña en texto plano del usuario semilla **admin@artisync.com** (solo su hash en la migración Flyway), las cuatro operaciones administrativas devolvieron consistentemente **403 Forbidden** — una respuesta correcta del control de acceso del backend, no un fallo del frontend. La prueba se repitió además desde un navegador real (Chromium automatizado con Playwright), capturando el tráfico de red completo en formato HAR, con el mismo resultado. De paso, esta ronda confirmó que la corrección de /admin/db-permisos/crear-usuario-sistema a POST /admin/usuarios sí llegó a un endpoint real (la respuesta pasó de 404 a 403), y reveló un segundo defecto de contrato, no relacionado, en **crud-usuarios.html**: el cambio de estado envía el campo **nuevoEstado** como texto cuando el backend espera **estadoCuenta** como booleano. Este trabajo extiende el Primer Corte al añadir y verificar la capa de consumo sobre el backend ya construido, y deja como trabajo futuro corregir ese payload de PATCH .../estado y repetir la prueba administrativa completa una vez se cuente con credenciales reales de un usuario **ADMIN**. Por último, en línea con lo declarado en la Introducción, este equipo utilizó asistencia de inteligencia artificial como herramienta de apoyo durante el desarrollo del frontend y la redacción de este informe, bajo supervisión y verificación humana de cada fragmento de código y cada afirmación técnica presentada.

Referencias

- [1] T. Bray. The javascript object notation (json) data interchange format. Request for Comments RFC 8259, Internet Engineering Task Force (IETF), 2017.
- [2] R. T. Fielding, M. Nottingham, and J. Reschke. Http semantics. Request for Comments RFC 9110, Internet Engineering Task Force (IETF), 2022.
- [3] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [4] ISO/IEC. Iso/iec 25010:2011 – systems and software engineering – systems and software quality requirements and evaluation (square) – system and software quality models. International Organization for Standardization, 2011.
- [5] M. B. Jones, J. Bradley, and N. Sakimura. Json web token (jwt). Request for Comments RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [6] OWASP Foundation. Owasp top 10:2021 – the ten most critical web application security risks. Open Worldwide Application Security Project, 2021.
- [7] H. Washizaki, editor. *SWEBOK Guide v4.0: Guide to the Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.